

Universidad Politécnica de Pachuca

DIRECCIÓN DE INGENIERÍA EN SOFTWARE

GUÍA DE ESTUDIO Y GUIÓN DE EXPOSICIÓN ORAL

DEFENSA TÉCNICA DEL PROYECTO
INTEGRADOR:

**Xiú — Sistema de Reservas SOA
y Microservicios**

PROFESORA: M. en C. Jazmín Rodríguez Flores

INTEGRANTES:

- Cristopher Lopez Suarez
- Jovanny Hernandez Hernandez

ASIGNATURA: Arquitectura Orientada a Servicios (AOS)

PROPÓSITO: Preparación para la evaluación oral, guiones para los videos de demostración (Cliente y Servidor) y explicación técnica exhaustiva del código fuente.

0. Índice

1. Guiones de Grabación para los Videos Demostrativos	2
1.1. Video 1: Demostración y Explicación del Cliente Funcional (Vercel SPA + GitHub)	2
1.2. Video 2: Demostración y Explicación del Servidor Funcional (Railway NestJS + Render FastAPI)	3
2. Simulacro de Examen Oral: Preguntas y Respuestas Clave de la Maestra	3
3. Explicación Detallada del Código Fuente Archivo por Archivo	5
3.1. Estructura del Proyecto en el Sistema de Archivos	5
3.2. Archivos Clave del FRONTEND (React 18 SPA)	5
3.3. Archivos Clave del BACKEND 1 (NestJS Auth-Service)	6
3.4. Archivos Clave del BACKEND 2 (FastAPI Menu-Service)	6
4. Resumen de Puntos de Defensa Rápida para el Examen	7

1. Guiones de Grabación para los Videos Demostrativos

La maestra evaluará el funcionamiento práctico a través de demostraciones en video. A continuación tienes los guiones explicativos paso a paso y palabra por palabra que debes pronunciar durante la grabación de pantalla.

1.1. Video 1: Demostración y Explicación del Cliente Funcional (Vercel SPA + GitHub)

Guión Explicativo para el Video del CLIENTE (Paso a Paso)

Paso 1: Introducción y Repositorio en GitHub (0:00 – 0:30)

"Hola profesora, en este video demostramos la funcionalidad del programa cliente de nuestro sistema Xiú. El código fuente está alojado en nuestro repositorio público de GitHub <https://github.com/Crisls24/Restaurante-.git>, donde se puede observar la colaboración entre Christopher López y Jovanny Hernández mediante la rama principal main. El frontend fue desarrollado utilizando React 18 con Vite y Tailwind CSS, y se encuentra desplegado de manera continua en la plataforma Vercel en la URL oficial <https://restaurante-frontend-omega.vercel.app/>."

Paso 2: Navegación del Cliente Público y Reservación (0:30 – 1:30)

"Como cliente público, el usuario ingresa a la vista principal donde observa la presentación del restaurante. Al dar clic en 'Ver Menú', el cliente consume asíncronamente los datos del microservicio NoSQL en Render. Posteriormente, el cliente hace clic en 'Reservar Mesa', donde se despliega un formulario interactivo. Al seleccionar fecha, hora y número de personas, la SPA ejecuta una petición POST al microservicio de reservaciones. El cliente recibe la respuesta en JSON, muestra el comprobante digital en pantalla e incluye un botón directo para enviar la confirmación vía WhatsApp Web."

Paso 3: Inicio de Sesión y Roles (1:30 – 2:30)

"Para el personal del restaurante, el cliente incluye la pantalla de Login en /login. Al ingresar las credenciales de Administrador (admin@xiu.mx) o Mesero (mesero@xiu.mx), la SPA realiza la autenticación contra el backend, guarda el token JWT en el localStorage y habilita el menú protegido en tiempo real. Aquí mostramos el Panel de Control con indicadores, el Tablero de Mesas con estados visuales (verde=libre, rojo=ocupada), la Gestión de Menú para crear y borrar platillos, y el módulo de Reportes estadísticos."

1.2. Video 2: Demostración y Explicación del Servidor Funcional (Railway NestJS + Render FastAPI)

Guión Explicativo para el Video del SERVIDOR (Paso a Paso)

Paso 1: Arquitectura SOA y Despliegue en la Nube (0:00 – 0:40)

"Buenas tardes profesora, en este video demostramos el funcionamiento del servidor. Nuestra arquitectura servidor está orientada a servicios (SOA) y está dividida en dos microservicios políglotas totalmente independientes desuscriptos de la capa visual:"

1. **Auth-Service (NestJS/TypeScript):** Desplegado en Railway en la URL <https://restaurante-production-36c3.up.railway.app/api>, encargado de la autenticación JWT, usuarios y reservaciones en la base de datos relacional MySQL.
2. **Menu-Service (FastAPI/Python):** Desplegado en Render en la URL <https://menu-service-u810.onrender.com/api>, encargado del catálogo NoSQL en MongoDB Atlas.

Paso 2: Demostración de Endpoints REST en Vivo (0:40 – 2:00)

"Demostramos el funcionamiento del Auth-Service ejecutando una petición POST a `/api/auth/login` con las credenciales de admin. El servidor responde HTTP 201 enviando el token JWT firmado con el algoritmo HS256. Al consultar GET `/api/reservations/available`, el servidor valida la disponibilidad de las 12 mesas en MySQL y retorna el JSON. Asimismo, demostramos la seguridad: si intentamos consultar un endpoint protegido sin el encabezado Authorization Bearer, el Guard de NestJS intercepta la petición y responde HTTP 401 Unauthorized."

Paso 3: Demostración del Servicio NoSQL en FastAPI (2:00 – 3:00)

"Por parte del Menu-Service en Python FastAPI, ejecutamos la petición GET `/api/menu`. El servidor conecta de forma asíncrona mediante la librería Motor a MongoDB Atlas y retorna los 10 documentos en formato JSON. Cuando el administrador crea un nuevo platillo con POST `/api/menu`, FastAPI valida el esquema Pydantic e inserta el nuevo documento en la colección de MongoDB en tiempo real."

2. Simulacro de Examen Oral: Preguntas y Respuestas Clave de la Maestra

Si la profesora te realiza preguntas durante la evaluación oral, estas son las preguntas más probables y la respuesta técnica exacta que debes dar para obtener la calificación máxima:

Pregunta 1: ¿Por qué eligieron una Arquitectura Orientada a Servicios (SOA) y no un sistema monolítico?

Respuesta Exacta:

Eligimos SOA porque nos permite desacoplar los componentes del sistema. El frontend en React no depende del lenguaje del backend; puede consumir datos de NestJS en TypeScript y de FastAPI en Python simultáneamente. Además, SOA brinda escalabilidad independiente: si la demanda de reservaciones aumenta, podemos escalar

el servicio de autenticación en Railway sin necesidad de duplicar el servicio de menú en Render."

Pregunta 2: ¿Por qué usaron Persistencia Políglota (MySQL + MongoDB)? ¿Por qué el menú en NoSQL y las reservas en Relacional?

Respuesta Exacta:

Usamos MySQL para el módulo de usuarios y reservaciones porque requerimos propiedades ACID, integridad referencial y relaciones estrictas mediante llaves foráneas entre la tabla de mesas y reservaciones. En cambio, para el catálogo del menú utilizamos MongoDB (NoSQL) porque los platillos tienen un esquema flexible; un platillo puede incluir etiquetas dinámicas de alérgenos, promociones o ingredientes sin necesidad de alterar una estructura de tabla rígida."

Pregunta 3: ¿Cómo funciona la seguridad y cómo evitan que un usuario no autorizado modifique las mesas o el menú?

Respuesta Exacta:

*"La seguridad se implementó mediante tokens JWT (JSON Web Tokens) sin estado. Cuando el usuario inicia sesión, el servidor valida el hash bcrypt de la contraseña y retorna el JWT. En las peticiones posteriores, el cliente envía el token en el encabezado **Authorization: Bearer <token>**. En NestJS, la clase **JwtAuthGuard** e **RolesGuard** interceptan la petición antes de llegar al controlador. Si el token no es válido o el usuario no tiene el rol **admin**, el Guard bloquea el acceso retornando un error 401 o 403."*

Pregunta 4: ¿Qué es un DTO y cómo lo utilizan en su código NestJS?

Respuesta Exacta:

*Un DTO (Data Transfer Object) es una clase que define la estructura exacta y las reglas de validación de los datos enviados desde el cliente. En NestJS creamos la clase **CreateReservationDto** utilizando decoradores de **class-validator** como **@IsString()**, **@Matches()** y **@Min()**. El **ValidationPipe** global revisa el cuerpo del JSON antes de entrar al controlador; si algún campo no cumple el formato (por ejemplo la fecha), el servidor responde un error 400 Bad Request detallado."*

Pregunta 5: ¿Por qué al consultar el menú en Render la primera petición tarda unos segundos?

Respuesta Exacta:

"Esto se debe al mecanismo de 'Cold Start' (arranque en frío) de los servicios gratuitos de Render. Cuando el servicio no recibe tráfico durante 15 minutos, Render suspende temporalmente el contenedor en la nube para ahorrar recursos. Al recibir la primera petición, el contenedor se vuelve a iniciar en aproximadamente 25-30 segundos y responde HTTP 200 OK. Las peticiones subsecuentes responden de inmediato."

Pregunta 6: ¿Cómo colaboraron en GitHub y qué estructura de control de versiones utilizaron?

Respuesta Exacta:

"Gestionamos el proyecto en el repositorio público <https://github.com/Crisls24/Restaurante>. Christopher trabajó en los módulos del servidor NestJS, TypeORM y autenticación, mientras que Jovanny trabajó en el frontend React, el microservicio FastAPI NoSQL y la integración de notificaciones. Sincronizamos nuestros cambios mediante com-

mits claros en la rama principal, integrando despliegue continuo (CI/CD) automatizado hacia Vercel, Railway y Render."

3. Explicación Detallada del Código Fuente Archivo por Archivo

A continuación se desglosa el propósito técnico de cada archivo relevante de la estructura del proyecto para que puedas responder cualquier consulta específica sobre el código.

3.1. Estructura del Proyecto en el Sistema de Archivos

```

1 Restaurante/
2 |-- frontend/                                # Aplicacion SPA React (Vercel)
3 |   |-- src/
4 |   |   |-- components/ Navbar.jsx, TableBoard.jsx, Footer.jsx
5 |   |   |-- pages/ Login.jsx, Reservation.jsx, AdminMenu.jsx, Reports.
6 |   |   |-- services/ api.js (Axios Instance)
7 |   |   +-- App.jsx (React Router DOM)
8 +-- services/
9 |   |-- auth-service/                        # Microservicio NestJS/MySQL (
10 |   |   |-- src/
11 |   |   |   |-- auth/ auth.controller.ts, auth.service.ts, jwt.
12 |   |   |   |-- reservations/ reservations.controller.ts, reservations
13 |   |   |   |   |-- dto/ create-reservation.dto.ts
14 |   |   +-- menu-service/                    # Microservicio FastAPI/MongoDB (
15 |   |   |   |-- app/
16 |   |   |   |   |-- main.py (FastAPI App & CORS)
17 |   |   |   |   |-- database.py (Motor Async MongoDB Connection)
18 |   |   |   +-- routers/ menu.py (CRUD Endpoints)

```

3.2. Archivos Clave del FRONTEND (React 18 SPA)

1. **src/App.jsx:** Archivo principal de enrutamiento. Utiliza `BrowserRouter`, `Routes` y `Route` de React Router DOM v6 para renderizar componentes dinámicos sin recargar la página.
2. **src/pages/Login.jsx:** Renderiza el formulario de acceso para empleados. Captura email y contraseña, ejecuta la petición a `VITE_AUTH_API/auth/login`, guarda la respuesta en `localStorage.setItem('token', data.token)` y actualiza el estado global del usuario.
3. **src/pages/Reservation.jsx:** Formulario para que los clientes registren sus reservas. Incluye un selector de hora/fecha y calcula la disponibilidad llamando a `GET /reservations/available`. Al enviar, genera el comprobante y arma la URL dinámica de WhatsApp `https://wa.me/5217712430474?text=...`

4. **src/pages/TableBoard.jsx**: Componente interactivo que dibuja la cuadrícula de las 12 mesas del restaurante. Aplica clases CSS dinámicas según el estado devuelto por el servidor (**libre=verde**, **ocupada=rojo**, **reservada=amarillo**).
5. **src/pages/AdminMenu.jsx**: Interfaz exclusiva de Administrador que consume el microservicio Python FastAPI en Render para agregar, editar y eliminar platillos de MongoDB Atlas mediante modales de React.

3.3. Archivos Clave del BACKEND 1 (NestJS Auth-Service)

1. **src/auth/auth.service.ts**: Contiene la lógica del inicio de sesión. Utiliza `bcrypt.compare()` para comparar la contraseña ingresada contra el hash de la base de datos MySQL. Si es correcta, invoca `jwtService.sign()` con el payload del usuario.
2. **src/auth/jwt.strategy.ts**: Estrategia de Passport JWT. Extrae el token enviado en la cabecera HTTP `Authorization: Bearer <token>`, valida la firma contra la clave secreta `JWT_SECRET` y adjunta los datos del usuario al objeto `request.user`.
3. **src/reservations/dto/create-reservation.dto.ts**: Clase DTO provista de reglas de validación stictas de NestJS. Ejemplo de código:

```
1 export class CreateReservationDto {
2   @IsString()
3   cliente_nombre: string;
4
5   @IsString()
6   cliente_telefono: string;
7
8   @IsString()
9   @Matches(/^\\d{4}-\\d{2}-\\d{2}$/)
10  fecha: string;
11
12  @IsString()
13  @Matches(/^([01]\\d|2[0-3]):[0-5]\\d$/)
14  hora_inicio: string;
15
16  @IsNumber()
17  @Min(1)
18  @Max(20)
19  num_personas: number;
20 }
```

4. **src/reservations/reservations.controller.ts**: Define las rutas HTTP para reservaciones. Incluye el método `create()` decorado con `@Post()` y métodos protegidos con `@UseGuards(JwtAuthGuard)`.

3.4. Archivos Clave del BACKEND 2 (FastAPI Menu-Service)

1. **app/main.py**: Punto de entrada del microservicio en Python. Instancia la clase `FastAPI()`, añade el middleware `CORSMiddleware` para permitir origen cruzado desde Vercel e incluye las rutas del enrutador de menú.
2. **app/database.py**: Utiliza la librería cliente `motor.motor_asyncio` para establecer una conexión no bloqueante hacia el cluster de MongoDB Atlas usando la cadena de conexión `MONGO_DETAILS`.

3. **app/routers/menu.py:** Contiene las funciones asíncronas `async def get_menu()`, `async def create_menu_item()` y `async def delete_menu_item()` que ejecutan operaciones CRUD directamente sobre la colección `menu_items` de MongoDB.

4. Resumen de Puntos de Defensa Rápida para el Examen

Si la profesora te pide hacer una síntesis express del proyecto en 1 minuto, di exactamente esto:

Resumen Express de 1 Minuto para el Examen

*"Nuestro proyecto **Xiú** es un sistema restauranero basado en **Arquitectura Orientada a Servicios (SOA)** con persistencia políglota y despliegue distribuido en la nube."*

1. **Frontend SPA:** Desarrollado en React 18 responsivo, desplegado en **Vercel**.
2. **Auth & Reservaciones:** Microservicio NestJS en TypeScript con persistencia relacional **MySQL**, desplegado en **Railway** con autenticación **JWT**.
3. **Catálogo de Menú:** Microservicio FastAPI en Python 3.11 con persistencia NoSQL **MongoDB Atlas**, desplegado en **Render**.
4. **Control de Versiones y Pruebas:** Desarrollado colaborativamente en **GitHub** con una suite de pruebas E2E aprobada al 100 %.